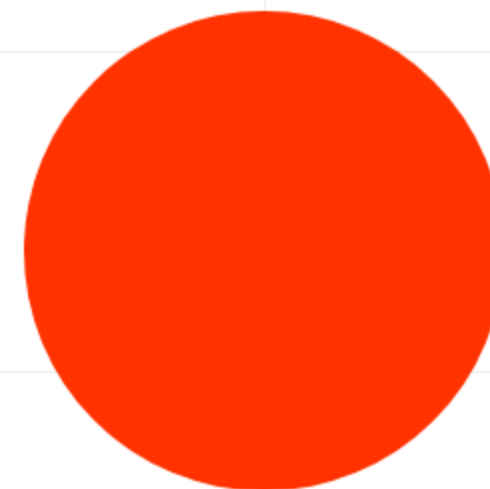


02

GIT ROADMAP

2026 · LECTURE 02



东北大学悉尼智能科技学院 · 实习课程

Git 版本控制

—— 从此告别"最终版_v3_改_最终版.docx"

代码的"时光机": 每一次改动都被精确记录, 随时回到任意历史版本。

INSTRUCTOR · 讲师

靳老师

北京科迪诺亿科技有限公司
东北大学秦皇岛分校

01 / 30



翻页



编辑



全屏



你的桌面是不是这样

几乎所有写过文档的人，都干过这件事 —— 用文件名当版本号。

- | | |
|------------------------|-------|
| ▶ 毕业论文.doc | 初稿 |
| ▶ 毕业论文_修改版.doc | 第一轮反馈 |
| ▶ 毕业论文_最终版.doc | 以为定稿 |
| ▶ 毕业论文_最终版2.doc | 又改了一次 |
| ▶ 毕业论文_打死不改版.doc | 崩溃边缘 |
| ▶ 毕业论文_打死不改版_老师让改版.doc | 投降 |

互动提问 ▶ 有没有人这样命名过文件？你最后怎么找到"对的那个版本"的？

为什么"文件名后缀"不靠谱

靠手动命名管理版本，迟早会遇到这五个坑。

- **想回到上一版** —— 找不到对应的文件了，被覆盖或删除
- **多人协作冲突** —— A 改了 v2，B 基于 v1 改，怎么合并？
- **改坏了没备份** —— 一行错代码毁掉整个项目，从头再来
- **磁盘空间爆满** —— 十几个版本副本占满硬盘
- **不知道改了啥** —— 看着两份文档，分不清区别在哪

结论 ▶ 文档越写越多、人越凑越齐，必须有专业工具来管版本。

Git = 代码的“时光机”

Git 是一个版本控制工具，帮你自动记录每一次改动，并在需要时随时穿越回去。

01

RECORD

记录每次修改

谁改的、什么时候改的、改了什么内容，全自动留痕。

02

REWIND

回到任意版本

三个月前的某次提交，一条命令就能找回来。

03

COLLAB

多人协作合并

同时修改，自动合并；冲突时明确提示该怎么处理。

一句话记住 ▶ Git 把“我的代码改坏了怎么办”这件恐怖的事，变得完全不恐怖。

04

GIT TO THE RESCUE

几乎所有程序员都在用

学会 Git 不是"加分项"，而是进入企业开发的基本门槛。

领域	代表项目 / 公司
互联网大厂	腾讯、阿里、字节跳动、美团、京东 —— 全员 Git
开源项目	Linux 内核、Android 系统、Spring 框架、Vue / React
创业公司	从 3 人小团队到 300 人中厂，Git 是标配
外企 / 远程	GitHub 是全球程序员的"普通话"
科研 / 论文	越来越多论文配套代码、LaTeX 源码托管在 Git

结论 ▶ 学会 Git = 具备企业开发的基本技能，写代码的"入场券"。

Git 的三个区域

理解这三个区域，就理解了 Git 80% 的工作原理。用"购物"做类比最容易记住。

01

WORKING DIRECTORY

工作区

你正在编辑代码的文件夹，就是你的项目目录。相当于"超市货架"。

02

STAGING AREA

暂存区

改好但还没正式保存的改动。相当于"购物车"——选好了，但还没结账。

03

REPOSITORY

仓库

Git 永久保存所有版本的地方。相当于"已下单的商品清单"，再也不丢。

06

THREE AREAS

从工作区到仓库的两步

写代码 → 暂存 → 提交，整个过程就像在超市"选货 → 放购物车 → 结账下单"。



关键点 ▶ 每次 commit 都会生成一个"版本快照", Git 会永远记住这一刻的状态。

一个 Commit 包含四样东西

提交不是简单的"保存", 而是一次完整的、可追溯的版本记录。



改了哪些

具体哪些文件、新增 / 修改 / 删除了哪些行。



谁改的

作者姓名 + 邮箱, 团队协作时用来追责和感谢。



什么时候

精确到秒的时间戳, 可以按时间回溯历史。



说明了啥

commit message —— 一句话说清"这次改了什么"。

GOOD MESSAGE

好例子

- 添加用户登录功能
- 修复成绩查询报错问题
- 优化首页加载速度

BAD MESSAGE

差例子

- 修改
- update
- 111 / 啥也没写

分支 = 独立的开发线路

从主线分出来一条独立的小路，互不影响，最后再合回去。

ANALOGY

写论文的类比

- 主线 (main) = 论文正文
- 分支 = 实验数据章节
- 两个章节独立写，最后合订

WHY BRANCH

为什么要用分支

- 开发新功能时不影响主线
- 多人各开发各的，互不干扰
- 实验性改动可随时丢弃

一句话 ▸ 没有分支，多人协作就是灾难；有了分支，每个人都能安心写自己的代码。

分支长这个样子

主线笔直往前走，feature 分支从某个点分叉出来，做完再合并回去。

第一步：装好 + 告诉 Git 你是谁

Git 安装完，第一件事是配置用户信息 —— 因为每个 commit 都要署名。

- 下载安装 —— 官网 git-scm.com，一路下一步
- 验证安装 —— `git --version` 看版本号
- 配置姓名 —— 让 commit 署上你的名字
- 配置邮箱 —— 团队协作时用来联系你
- 只需配一次 —— `--global` 全局生效

BASH · 安装后首次配置

SHELL

1. 验证安装成功

```
git --version
```

2. 配置你的身份（只需一次）

```
git config --global user.name "靳老师"
```

```
git config --global user.email "jin@example.com"
```

3. 检查配置是否生效

```
git config --list
```

让一个文件夹变成 Git 仓库

两种方式：自己新建 / 从远程克隆。先看自己新建。

BASH · 创建第一个仓库

SHELL

1. 新建一个项目文件夹

```
mkdir my-project
```

```
cd my-project
```

2. 让它变成 Git 仓库

```
git init
```

输出:

```
# Initialized empty Git repository in /Users/you/my-project/.git/
```

注意 ▶ 执行后文件夹里会多一个隐藏的 .git 目录 —— 这就是 Git 的"记忆库", 千万别手动删它!

— 另一种方式 —— git clone <地址> 从远程复制已有仓库（后面会讲）

12

GIT INIT

随时问 Git: "现在啥情况"

git status 是用得最多的命令 —— 像一面镜子，照出文件当前处于哪种状态。

STATE 01 未跟踪 新文件，Git 还不认识。 显示为 Untracked 红色。	STATE 02 已暂存 git add 过了，等着 commit。 显示为 Staged 绿色。	STATE 03 已提交 已经 commit，安全归档。 显示 nothing to commit。
---	--	---

养成习惯 ▶ 每次想 commit 之前，先 git status 看一眼 —— 不会错过任何改动。

把改动加入"购物车"

git add 把工作区的改动放进暂存区，告诉 Git: "这些改动我打算提交了。"

- 添加单个文件 —— git add README.md
- 添加所有改动 —— git add . (最常用)
- 添加某类文件 —— git add *.java
- 不会立即生效 —— 还要 git commit 才正式归档

BASH · ADD 的几种姿势

SHELL

添加单个文件

```
git add README.md
```

添加所有改动 (常用)

```
git add .
```

添加所有 .java 文件

```
git add *.java
```

类比 ▶ 把商品加入购物车，结账前还能反悔 —— git reset 可以撤掉暂存。

点击"下单"的那一下

git commit 把暂存区的所有改动正式归档，生成一个永久版本。

BASH · 第一次提交

SHELL

1. 先把改动加入暂存区

```
git add .
```

2. 提交，附上说明文字

```
git commit -m "初始化项目，添加 README 文件"
```

输出：

```
# [main (root-commit) abc1234] 初始化项目...
```

```
# 1 file changed, 1 insertion(+)
```

— **-m 参数** —— 后面跟一句话说明，是 commit message

— **没有 add 的改动** —— 不会被 commit！只提交暂存区里的

15

第一次提交叫 **root-commit** —— 整个仓库历史的起点

从零走完一遍完整流程

这七步是 Git 日常 90% 的工作，背下来就能干活。

BASH · 第一次完整提交

SHELL

1. 创建仓库

```
git init
```

2. 写一个文件

```
echo "Hello World" > hello.txt
```

3. 查看状态 (hello.txt 是 Untracked)

```
git status
```

4. 加入暂存区

```
git add hello.txt
```

5. 再次查看状态 (hello.txt 是 Staged)

```
git status
```

6. 提交

```
git commit -m "创建 hello.txt 文件"
```

7. 再次查看状态 (nothing to commit)

```
git status
```

16

FULL WALKTHROUGH

翻看版本历史笔记本

git log 按时间倒序列出所有 commit，像一本完整的开发日志。

BASH · 查看完整历史

SHELL

```
git log

# 默认输出（按时间倒序）：
commit abc1234 (HEAD -> main)
Author: 靳老师 <jin@example.com>
Date:   Wed Jun 19 14:30:00 2026

    创建 hello.txt 文件

commit def5678
...
```

— git log --oneline —— 简洁模式，一行一条，看大势最爽

17 commit 哈希值 —— 每个 commit 的唯一 ID (abc1234)

GIT LOG 退出 log —— 按小写 q 即可

对比"改了啥"

git diff 把两份代码放在一起，标出哪行加了、哪行删了。

- **git diff** —— 工作区 vs 暂存区（还没 add 的改动）
- **git diff --staged** —— 暂存区 vs 仓库（add 了还没 commit）
- **红色 -** —— 删除的行
- **绿色 +** —— 新增的行

类比 ▶ 老师批改作文，红笔标出哪里改了。

BASH · DIFF 输出示例

DIFF

```
git diff
```

```
# 输出:
```

```
diff --git a/hello.txt  
      b/hello.txt  
index abc1234..def5678
```

```
-Hello World  
+Hello Git!
```

后悔了怎么办

三种"反悔"姿势，破坏力递增 —— 动手前先想清楚。

命令	作用	危险度
git restore <文件>	丢弃工作区的修改（还没 add 的）	⚠ 中
git reset --soft <commit>	撤销 commit，改动保留在暂存区	低
git reset --hard <commit>	彻底回到某个版本，丢弃所有改动	🔴 高

BASH · 三种用法SHELL

```
# 丢弃单个文件的修改（还没 add）
git restore hello.txt

# 撤销最后一次 commit，但代码不丢
git reset --soft HEAD^

# 彻底回到上一个版本（危险！代码会丢！）
git reset --hard HEAD^
```

19 重要提醒 ▶ --hard 是不可逆的核武器 —— 没提交的改动会全部消失，慎用！

有些文件不该进仓库

编译产物、IDE 配置、临时文件 —— 用 .gitignore 告诉 Git "这些别管"。

- 编译产物 —— target/、build/
- IDE 配置 —— .idea/、.vscode/
- 依赖目录 —— node_modules/
- 临时文件 —— *.log、*.tmp
- 敏感信息 —— .env、密钥文件

.GITIGNORE · 项目根目录

GITIGNORE

IDE 配置

.idea/

.vscode/

Java 编译产物

target/

*.class

日志和临时文件

*.log

*.tmp

系统文件

.DS_Store

Thumbs.db

分支的四件套

查、建、切、合 —— 团队协作每天都在用的四个动作。

命令	作用
<code>git branch</code>	查看所有分支（当前分支前有 *）
<code>git branch <名></code>	创建分支（但不切换过去）
<code>git checkout <名></code>	切换到指定分支（也可用 <code>git switch</code> ）
<code>git checkout -b <名></code>	创建 + 切换，一步到位（最常用）

BASH · 分支操作示例SHELL

```
# 看看当前有哪些分支
git branch
# * main

# 创建并切到新分支
git checkout -b feature-login
# Switched to a new branch 'feature-login'
```


完整走一遍分支开发流程

从创建分支到合并回主线——团队协作的标准动作。

BASH · 完整分支流程

SHELL

1. 创建并切换到新分支

```
git checkout -b feature-login
```

2. 在新分支上写代码

```
echo "登录功能" > login.java
```

3. 提交

```
git add .
```

```
git commit -m "添加登录功能"
```

4. 切回主分支

```
git checkout main
```

5. 合并新分支的成果

```
git merge feature-login
```

类比 像是在草稿纸上写好实验数据，确认无误后抄到正文里。

22

BRANCH IN ACTION

两人改了同一行代码

Git 不知道保留哪个 —— 这就是"冲突"。看懂冲突标记，三步解决。

```
<<<<<< HEAD 这是主分支 (main) 的内容 ===== 这是新分支 (feature) 的内容 >>>>>> feature-login
```

01 打开冲突文件，手动选择保留哪部分（或合并两者）

02 删除三组标记：<<<<<< / ===== / >>>>>>

03 `git add <文件>` → `git commit -m "解决冲突"` 完成合并

经验 ▶ 冲突不可怕，可怕的是不看 diff 直接覆盖 —— 先沟通再合并。

把代码放到云端

本地仓库只是你电脑上的一个文件夹。远程仓库才是团队协作的中心。

RECOMMENDED

Gitee 码云

- 国内访问快，无需翻墙
- 中文界面，对新手友好
- 免费私有仓库不限数量
- 网址：gitee.com

ALTERNATIVE

GitHub

- 全球最大开源社区
- 国外平台，国内访问不稳定
- 适合看开源项目源码
- 网址：github.com

行动项 ▶ 课后立即注册一个 Gitee 账号 —— 下半节课实操要用。

本地代码上传到云端

第一次推送三步走，之后只需 `git push` 一条命令。

BASH · 首次推送

SHELL

1. 关联远程仓库（只需一次）

```
git remote add origin https://gitee.com/yourname/my-project.git
```

2. 第一次推送（-u 记住关联）

```
git push -u origin main
```

3. 以后只需这一条

```
git push
```

— **origin** —— 远程仓库的别名（约定俗成）

— **-u 参数** —— 把本地 main 和远程 main 关联起来，以后省略

— **类比** —— 把写好的作业上传到老师的网盘

25

GIT PUSH

云端代码下载到本地

git pull 把远程最新代码同步下来 —— push 的反向操作。

- 别人推了新代码 —— 你 pull 一下就同步
- 换电脑继续开发 —— pull 拉到最新进度
- push 被拒绝时 —— 先 pull 再 push（远程有新提交）

类比 ▶ 从网盘下载最新的作业要求。

BASH · 拉取远程更新

SHELL

每天开工第一件事

git pull

输出:

Updating abc1234..def5678

Fast-forward

login.java | 12 ++++++

1 file changed, 12 insertions(+)

真实团队每天的协作循环

从 clone 到 push，这是企业开发最标准的四步。

01

Clone 远程仓库

第一次拿到项目代码

```
git clone <仓库地址>
```

02

创建分支开发

在自己的分支上干活

```
git checkout -b feature-x
```

03

提交并推送

commit + push 到远程

```
git push origin feature-x
```

04

合并到主分支

提交 PR 或本地 merge

```
git merge feature-x
```

循环往复 ▶ 每天上班就重复这四步，企业开发的日常。

27

TEAM WORKFLOW

今天学到了什么

五个核心知识点，回去再回顾一遍。

01

为什么需要版本控制

告别"文件名当版本号"，让每一次改动都被精确记录。

03

核心命令

init / add / commit / status / log —— 日常五件套。

05

远程仓库

push 推到云端，pull 拉到本地，团队协作的中心。

02

三个区域

工作区 → 暂存区 → 仓库，理解这三步就理解了 Git。

04

分支

独立开发、最后合并，多人协作的基础设施。

06

协作流程

clone → 建分支 → push → 合并，企业开发每日循环。

贴在桌前的命令速查表

11 个命令覆盖日常 95% 的工作。

命令	作用
git init	把当前文件夹变成 Git 仓库
git clone <地址>	从远程复制一个已有仓库
git status	查看当前仓库状态（最常用）
git add .	把所有改动加入暂存区
git commit -m "说明"	把暂存区的改动正式提交
git log --oneline	简洁模式查看历史
git branch	查看所有分支
git checkout -b <名>	创建并切换分支
git merge <分支>	合并指定分支到当前分支
git push	推送本地代码到远程仓库
git pull	从远程拉取最新代码

下半节课我们做什么

理论结束，开始动手 —— 三件事，每件都重要。

NEXT

Git 实操练习

从安装配置到团队协作模拟，把今天学的所有命令亲手敲一遍。

▶ 内容包含：环境配置与安装 · 个人独立练习 · 两人一组团队协作模拟

提前准备 ▶ 注册一个 Gitee 账号 (gitee.com)，下半节课直接开干！

30

NEXT SESSION